



SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

**AI BASED INDOOR ROBOT NAVIGATION USING A* SEARCH AND
DYNAMIC OBSTACLE AVOIDANCE**

A CAPSTONE PROJECT REPORT

*A Capstone Project Report submitted in partial fulfilment of the requirements
for the Course of*

CSA1709- Artificial Intelligence

to the award of the degree of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

V. MADHULIKA (192312620)

B. BHAVANA (192312428)

V. SIREESHA (192472208)

Under the Supervision of

Dr. P. Rajaram

SIMATS ENGINEERING

Saveetha Institute of Medical and Technical Sciences

Chennai-602105

SEPTEMBER 2026

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

BONAFIDE CERTIFICATE

This is to certify that the Capstone Project entitled “**AI BASED INDOOR ROBOT NAVIGATION USING A* SEARCH AND DYNAMIC OBSTACLE AVOIDANCE**” has been carried out by **V. MADHULIKA (192312620)**, **B. BHAVANA (192312428)**, **V. SIREESHA (192472208)** under the supervision of **Dr. P. Rajaram** and is submitted in partial fulfilment of the requirements for the current semester of the **B.E Electronics and Communication Engineering and B.E Computer Science and Artificial Intelligence Engineering** programme at Saveetha Institute of Medical and Technical Sciences, Chennai.

COURSE COORDINATOR

DR. KRISHNA KUMAR KATHIRESAN,
Professor,
CSE - Generative AI,
SIMATS Engineering ,
SIMATS, Chennai - 602105.

COURSE FACULTY

Dr .P .Rajaram,
Professor,
CSE/Data vista,
SIMATS Engineering,
SIMATS, Chennai - 602105.

Submitted for the Project Work Viva-Voce held on _____.

INTERNAL EXAMINER

EXTERNAL EXAMINER

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

DECLARATION

We, **V. MADHULIKA (192312620), B. BHAVANA (192312428), V. SIREESHA (192472208)**, of the Department of **B.E ECE and B.E CSE AI** SIMATS Engineering, Saveetha Institute of Medical and Technical Sciences, Chennai, hereby declare that the Capstone Project Work entitled “**AI BASED INDOOR ROBOT NAVIGATION USING A* SEARCH AND DYNAMIC OBSTACLE AVOIDANCE**” is the result of our own bonafide efforts. To the best of our knowledge, the work presented herein is original, accurate, and has been carried out in accordance with the principles of engineering ethics and academic integrity. All sources, references, data, and other materials used in the project have been appropriately acknowledged. We further declare that the data, results, analysis, and findings presented in this report have not been fabricated, falsified, or manipulated. Any external tools, software, datasets, computational resources, or AI-assisted tools used during the project have been appropriately disclosed. We confirm that all applicable ethical, academic, institutional, and professional requirements have been duly followed throughout the planning, execution, analysis, and documentation of the project.

Place: Chennai

Date:

Signature of the Students with Names

V. MADHULIKA (192312620)

B. BHAVANA (192312428)

V. SIREESHA (192472208)

SIMATS ENGINEERING
Saveetha Institute of Medical and Technical Sciences
Chennai-602105

INDIVIDUAL CONTRIBUTION STATEMENT

Student Name / Reg. No.	Specific Responsibilities	Design & Development Contribution	Testing & Analysis Contribution	Report Contribution	Approx. Contribution (%)
V. Madhulika (192312620)	A* search algorithm design, grid-based environment modelling, path-planning module	Implemented the A* search module (g(n), h(n), f(n) computation) and the grid-based environment representation	Tested static-obstacle path-planning scenarios (TC01, TC02, TC05, TC06)	Chapters 1, 3 (3.1–3.5)	35%
B. Bhavana (192312428)	Dynamic obstacle detection, replanning module, visualisation logic	Implemented the obstacle-detection and path-replanning modules and the map/robot visualisation	Tested dynamic-obstacle scenarios (TC03, TC04, TC07) and real-time response cases	Chapters 2, 4	30%
V. Sireesha (192472208)	Literature survey, requirement analysis, documentation and result evaluation	Assisted in module integration and tools/technology selection	Consolidated and analysed evaluation results across all test cases	Chapters 3 (3.6–3.7), 5, References, Appendices	35%
Total					100%

ABSTRACT

Indoor robot navigation is an important application of Artificial Intelligence and robotics, particularly in environments such as hospitals, offices, warehouses, laboratories, and homes. A central engineering challenge in this domain is enabling a robot to reach a specified destination efficiently while avoiding obstacles whose positions may change over time. Static, one-time path computation is insufficient for real indoor spaces because people, furniture, and other robots can move into a previously planned route.

This project proposes an AI-based Indoor Robot Navigation System that combines the A* search algorithm with a dynamic obstacle-avoidance mechanism. The indoor environment is represented as a grid-based map containing free cells, static obstacles, a start position and a goal position. The A* algorithm computes the total estimated cost of a candidate path as $f(n) = g(n) + h(n)$, where $g(n)$ is the actual movement cost from the start to the current node and $h(n)$ is a heuristic estimate of the remaining cost to the goal; this allows the search to move efficiently toward the destination rather than exploring the entire grid.

Because static planning alone cannot handle real indoor conditions, the system continuously monitors the robot's route for dynamic obstacles using sensor or simulated sensor data. When a dynamic obstacle is detected on the planned path, the affected cell is marked as blocked and the A* search is re-executed from the robot's current position, generating a new collision-free route. The overall workflow therefore proceeds through environment representation, path planning, path execution, obstacle monitoring, and path replanning until the goal is reached.

The system was evaluated using multiple indoor grid maps and obstacle conditions, including empty maps, static-obstacle maps, single and multiple dynamic obstacles, and fully blocked routes. Results show that the system reliably computes an efficient initial path, detects dynamic obstacles along the route, and successfully replans to reach the destination when an alternative path exists, while correctly reporting failure when no valid route remains. The project demonstrates how a classical heuristic search algorithm can be combined with reactive obstacle-avoidance logic to achieve safer and more adaptive indoor autonomous navigation, and provides a practical foundation for extensions such as LiDAR-based sensing, camera-based obstacle classification, and multi-robot coordination.

Keywords

Artificial Intelligence, Indoor Robot Navigation, A* Search Algorithm, Path Planning, Dynamic Obstacle Avoidance, Autonomous Robotics.

TABLE OF CONTENTS

S.No.	Title	Page No.
1	Bonafide Certificate	2
2	Declaration	3
3	Individual Contribution Statement	4
4	Abstract	5
5	List of Figures	8
6	List of Tables	9
7	List of Abbreviations / Symbols	10
8	CHAPTER 1: INTRODUCTION AND ENGINEERING PROBLEM	11-13
9	1.1 Background	11
10	1.2 Need for the Project	11
11	1.3 Problem Statement	12
12	1.4 Project Aim	12
13	1.5 Project Objectives	12
14	1.6 Scope	12
15	1.7 Expected Engineering Outcome	13
16	CHAPTER 2: LITERATURE REVIEW AND EXISTING SOLUTIONS	14-15
17	2.1 Review Method	14
18	2.2 Review of Existing Work	14
19	2.3 Comparative Analysis	15
20	2.4 Research / Engineering Gap	15
21	2.5 Proposed Contribution	15
22	CHAPTER 3: ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY	16-21
23	3.1 Requirements	16
24	3.2 Constraints & Applicable Standards	16
25	3.3 Design Specifications	17
26	3.4 Alternative Solutions & Evaluation	17
27	3.5 Selected Design & Detailed Design	18
28	3.6 Trade-off Analysis	19
29	3.7 Sustainability, Ethics, Safety, Risk & Security	20
30	CHAPTER 4: IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT	22-25
31	4.1 Development Approach & Resources	22
32	4.2 Hardware / Software Implementation & Modern Tools Used	22
33	4.3 Test Plan & Verification	23
34	4.4 Results	23
35	4.5 Data Analysis & Engineering Judgment	24
36	4.6 Comparison with Existing Solutions & Achievement of Objectives	24
37	4.7 Strengths & Limitations	25
38	4.8 Project Planning, Roles & Budget	25
39	CHAPTER 5: CONCLUSION, FUTURE WORK AND	27-28

	REFLECTION	
40	5.1 Conclusion	27
41	5.2 Future Work	27
42	5.3 Professional Learning & Individual Reflection	27
43	REFERENCES	29
44	APPENDICES	30-32
55	Capstone Project Outcome Mapping Sheet	33

LIST OF FIGURES

Figure No.	Figure Name	Page No.
Figure 1.1	System Architecture of the Proposed Navigation System	18
Figure 2.1	A* Path Planning on a Grid with Static Obstacles	18
Figure 3.1	Dynamic Obstacle Detection and Replanning Sequence	21
Figure 4.1	Simulated Indoor Environment – Real-Time Path Replanning	24

LIST OF TABLES

Table No.	Table Name	Page No.
Table 1	Comparative Analysis of Existing Navigation Approaches	15
Table 2	Stakeholder / User Requirements	16
Table 3	Functional & Non-Functional Requirements	16
Table 4	Constraints & Applicable Standards	17
Table 5	Design Specifications	18
Table 6	Alternative Solutions – Evaluation Matrix	19
Table 7	Trade-off Analysis	19
Table 8	Sustainability, Ethics, Safety & Security Evaluation	20
Table 9	<i>Tools and Technologies Used</i>	23
Table 10	Sample Test Cases and Results	25

LIST OF ABBREVIATIONS / SYMBOLS

Symbol	Description	Unit
AI	Artificial Intelligence	—
A*	A-Star Search Algorithm	—
$g(n)$	Actual movement cost from start node to node n	—
$h(n)$	Heuristic estimate of cost from node n to goal	—
$f(n)$	Total estimated path cost, $f(n) = g(n) + h(n)$	—
DWA	Dynamic Window Approach	—
SLAM	Simultaneous Localization and Mapping	—
LiDAR	Light Detection and Ranging	—
TC	Test Case	—
PEP 8	Python Enhancement Proposal 8 (coding style guide)	—

CHAPTER 1 INTRODUCTION AND ENGINEERING PROBLEM

1.1 Background

With the advancement of Artificial Intelligence, robotics has become an important area for developing autonomous systems capable of sensing their environment and making decisions without continuous human control. Indoor robot navigation is one of the important problems in autonomous robotics: a robot operating inside a building must determine its current location, identify obstacles, select a suitable route, and reach a specified destination safely.

Traditional navigation methods work well in environments where obstacles remain fixed, but indoor environments are usually dynamic — people walk across corridors, objects are moved, doors are opened or closed, and other robots may enter the robot's path. This project addresses the problem using the A* search algorithm for path planning, combined with a dynamic obstacle-avoidance mechanism for responding to changes in the environment. A* evaluates possible paths using $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the start to the current node and $h(n)$ is a heuristic estimate of the remaining cost to the destination. The robot first plans a path with A*; while moving, the environment is continuously monitored, and if a dynamic obstacle blocks the planned route, the robot recalculates an alternative route.

1.2 Need for the Project

Indoor navigation requires more than a single, fixed route. Several practical observations motivate the project:

- People continuously move inside buildings, and furniture or equipment may change position.
- Corridors and pathways may become temporarily blocked without warning.
- A static path cannot respond automatically to environmental changes, so a robot following an outdated path risks collision.
- Manual/tele-operated navigation requires continuous human supervision, which does not scale to multiple robots or continuous operation.
- Recalculating the route when conditions change measurably improves navigation reliability and safety.

These needs are shared by hospitals, warehouses, offices, laboratories and educational institutions — the direct stakeholders of an intelligent indoor navigation system — who require robots that can plan efficient paths and still adapt safely when the environment does not match the original plan.

1.3 Problem Statement

Indoor mobile robots must travel from a start position to a goal position through an environment that contains walls, furniture, static obstacles, and people or objects whose positions change over time. A path computed once, using only the obstacles known at planning time, is not sufficient because the environment can change after the path has been generated — for example, a person entering a corridor that the robot has already planned to use. The engineering problem is therefore twofold and the two parts interact: (1) compute an efficient, collision-free path through a partially known, grid-represented indoor environment, and (2) detect obstacles that appear after planning and safely revise the route in real time, without excessive recomputation cost, while still guaranteeing the robot reaches its destination whenever a valid route exists, and reports failure correctly when none does.

1.4 Project Aim

To design and implement an AI-based indoor robot navigation system that uses the A* search algorithm for efficient path planning on a grid-based map and incorporates a dynamic obstacle-avoidance mechanism that detects environmental changes and replans the route so the robot can reach its destination safely and efficiently.

1.5 Project Objectives

- To design an AI-based indoor robot navigation system.
- To represent an indoor environment using a grid-based map.
- To implement the A* search algorithm for path planning.
- To calculate an efficient route between a source and destination.
- To detect static and dynamic obstacles appearing along the robot's path.
- To implement a dynamic obstacle-avoidance mechanism.
- To recalculate the path when the original route becomes blocked.
- To reduce unnecessary movement and navigation errors.
- To test the system under different indoor obstacle conditions.
- To evaluate the effectiveness of the proposed navigation approach against defined test cases.

1.6 Scope

In-Scope

- Indoor grid-map representation of free space and obstacles.
- Source and destination selection.
- Static obstacle representation and A* path planning.
- Dynamic obstacle detection using sensor or simulated sensor data.
- Path replanning when the route is blocked.

- Robot movement simulation and visualization.
- Performance evaluation using multiple test cases.

Out-of-Scope

- Outdoor GPS-based navigation.
- Complex autonomous-vehicle navigation.
- Large-scale industrial robot deployment.
- Full Simultaneous Localization and Mapping (SLAM) implementation.
- Complete commercial-grade hardware implementation.
- Multi-robot coordination.

Assumptions and Boundary Conditions

- The indoor environment can be adequately represented as a discrete grid of uniform cell size.
- The robot's position and the positions of obstacles can be obtained (from simulation or sensor data) at a rate sufficient for timely replanning.
- Dynamic obstacles are treated as blocking cells once detected; their future trajectories are not predicted in the current scope.

1.7 Expected Engineering Outcome

The expected outcome of this capstone project is a working software prototype — an algorithm and simulation, extensible to a physical robot — comprising: a grid-based indoor-environment model; an A* path-planning module; an obstacle-detection module for static and dynamic obstacles; a replanning module that regenerates a path when the current route is blocked; and a visualization module that displays the map, the robot, the obstacles and the computed path. Collectively these modules demonstrate an autonomous navigation process that plans, executes, monitors and adapts, and that can be evaluated quantitatively against the test cases defined in Chapter 4.

CHAPTER 2 LITERATURE REVIEW AND EXISTING SOLUTIONS

2.1 Review Method

Relevant literature was identified from peer-reviewed journals and conference proceedings on mobile-robot path planning, indoor navigation, and obstacle avoidance, focusing on work published between 2021 and 2025 that used the A* algorithm, its variants, or closely related search and reactive-planning techniques. Thirteen sources were reviewed in detail (see References); they were compared on the basis of the planning algorithm used, whether the approach handles dynamic obstacles, and its suitability for indoor, grid-represented environments.

2.2 Review of Existing Work

Several studies have investigated improved variants of A* for dynamic environments. Zhang et al. (2021) proposed an improved A* algorithm for robot navigation in dynamic settings, modifying the heuristic and search strategy to reduce turning cost and better handle moving obstacles. Li et al. (2024) combined A* with the Dynamic Window Approach (DWA), using A* for global path generation and DWA for local, reactive obstacle avoidance — an approach conceptually close to the two-stage plan-then-replan strategy adopted in this project.

Bao et al. (2023) examined static and dynamic global path planning using an improved A* algorithm for mobile robots, while Yang and Teng (2022) proposed an enhanced dynamic-window approach combined with improved A* for mobile-robot path planning, both reporting improved adaptability to changing obstacle layouts compared with plain A*. Jiang et al. (2023) developed a hybrid path-planning algorithm for mobile robots that similarly integrates global search with local correction. Sun et al. (2024) proposed a multi-environment path-planning method based on an improved A* algorithm capable of operating across different map types, and Kabir and Watanobe (2024) presented an enhanced robot-motion block for the A* algorithm aimed specifically at improving planning efficiency for robotic path planning.

Work focused specifically on indoor settings includes Liu and Qiu (2021), who applied an improved A* algorithm to indoor mobile-robot path planning, and Khedr et al. (2021), who implemented indoor path planning for a tracked mobile robot using Dijkstra's algorithm within the Robot Operating System (ROS) — providing a useful baseline against which the efficiency of A* can be understood. Ran et al. (2021) studied scene-perception-based visual navigation for mobile robots indoors, illustrating an alternative, sensor-heavy approach to the same problem that this project addresses with a lighter grid-search method. Niloy et al. (2021) provided a broad review of critical design and control issues for indoor autonomous mobile robots.

2.3 Comparative Analysis

Existing Approach	Advantages	Limitations	Relevance to Proposed Work
Classical A* on static grid maps	Simple, complete, finds an optimal path for known obstacles	Cannot react to obstacles that appear after planning	Forms the core planner; extended here with dynamic replanning
Improved / weighted A* variants	Faster convergence, reduced node expansion in large maps	Weighting can trade off optimality for speed	Informs heuristic-tuning choices for the grid-based planner
A* + Dynamic Window Approach (DWA) fusion	Combines global path planning with local, reactive obstacle avoidance	Higher implementation and tuning complexity	Conceptually closest to the proposed replanning strategy
Vision/SLAM-based indoor navigation	Does not require a pre-built map; localises the robot online	Computationally heavier; needs camera/LiDAR hardware	Identified as a natural future extension (Section 5.2)

Table 1: Comparative Analysis of Existing Navigation Approaches

2.4 Research / Engineering Gap

Across the reviewed literature, two consistent gaps emerge. First, many implementations that improve A* purely for planning efficiency (weighted variants, multi-environment variants) do not natively incorporate a real-time obstacle-monitoring loop, and instead assume the obstacle map is fixed once planning begins. Second, approaches that do add reactive avoidance (A*+DWA fusions) generally add considerable implementation and tuning complexity that is not always justified for a bounded indoor-navigation prototype at capstone scale. There remains a need for a comparatively lightweight system that (a) uses a well-understood, provably-correct global planner (A*) for path efficiency, and (b) adds a simple, well-tested replanning loop that reacts to dynamic obstacles without the full complexity of a fused local planner.

2.5 Proposed Contribution

This project contributes a modular indoor navigation system that keeps the A* planner and the dynamic-obstacle response as separate, clearly interfaced modules (Section 3.5), which simplifies testing and evaluation relative to tightly fused approaches while still directly addressing the identified gap: the system detects dynamic obstacles along the currently executing path and triggers a fresh A* search from the robot's current position rather than continuing to follow an outdated route. This keeps the engineering solution close to the proven strengths of A* identified in the literature while adding the minimum reactive capability needed for realistic indoor operation.

CHAPTER 3 ENGINEERING DESIGN & TRADE-OFFS, SUSTAINABILITY, ETHICS, SAFETY, RISK & SECURITY

3.1 Requirements

Stakeholder / User Requirements

Stakeholder	Requirement
Robot developers / researchers	A modular, extensible planning and obstacle-avoidance codebase suitable for further experimentation
Hospitals	Reliable, collision-free transport of medicines, documents or supplies through corridors shared with people
Warehouses	Navigation between storage areas that adapts to moving workers and relocated goods
Offices	Safe delivery/assistance navigation through shared, semi-dynamic spaces
Educational institutions	A clear, demonstrable teaching example of AI search combined with reactive robotics

Table 2: Stakeholder / User Requirements

Functional & Non-Functional Requirements

Requirement Type	Description	Measurable Target
Functional	Represent the indoor environment as a grid of free cells and obstacles	Grid of configurable size (e.g. 10×10 to 15×15 cells)
Functional	Compute a path from start to goal using A*	Valid path returned whenever one exists
Functional	Detect a dynamic obstacle appearing on the active path	Detected within one monitoring cycle after appearance
Functional	Replan the route when the current path is blocked	New path generated and executed without manual intervention
Non-Functional	Path-planning response time	Path computed in well under 1 second on a standard laptop for grids up to 15×15
Non-Functional	Reliability	System correctly reports “no valid path” when the goal is unreachable, in 100% of such test cases
Non-Functional	Usability	Visualization clearly distinguishes robot, goal, static obstacles and dynamic obstacles

Table 3: Functional & Non-Functional Requirements

3.2 Constraints & Applicable Standards

Only constraints that genuinely apply to this simulation-based prototype are listed. The project is technically constrained by the choice of a discrete grid representation and by

available development time; it has no significant economic, environmental, or regulatory constraints at prototype stage, since it does not yet deploy on physical hardware.

Standard / Guideline	Requirement	How Applied in This Project
PEP 8 (Python coding style)	Consistent, readable code layout and naming	Followed throughout the planning, detection and visualization modules
Structured Software Development Lifecycle (SDLC)	Requirement analysis → design → implementation → testing → evaluation	Used as the project's development process (Section 4.1)
Modular design principle	Independent, loosely-coupled modules with clear interfaces	Environment, planning, detection, replanning and visualization kept as separate modules (Section 3.5)

Table 4: Constraints & Applicable Standards

3.3 Design Specifications

Parameter	Target / Requirement	Unit	Priority
Grid cell size	Configurable, uniform	$m \times m$ (logical cell)	High
Path-planning time	< 1	second	High
Obstacle-detection latency	1 monitoring cycle	cycle	High
Replanning success (reachable goal)	100	% of test cases	High
Path optimality	Minimum-cost path returned by A*	grid steps	Medium
Map size supported (prototype)	up to 15×15	cells	Medium

Table 5 : Design Specifications

3.4 Alternative Solutions & Evaluation

Alternative 1 – A* with on-demand replanning: the robot follows an A*-computed path and a fresh A* search is triggered only when a dynamic obstacle is detected on the active route.

Alternative 2 – A* fused with a Dynamic Window Approach (DWA) local planner: A* provides a global route while a continuously-running local controller adjusts velocity/heading to avoid obstacles in real time.

Alternative 3 – Dijkstra's algorithm with periodic full re-runs: the route is recomputed on a fixed timer regardless of whether an obstacle has actually appeared.

Criterion	Weight	A* only (static replanning on demand)	A* + DWA fusion	Dijkstra + periodic re-run
Performance (path optimality)	30%	High	High	High

Criterion	Weight	A* only (static replanning on demand)	A* + DWA fusion	Dijkstra + periodic re-run
Implementation complexity	25%	Low	High	Low
Real-time responsiveness	20%	Medium	High	Low
Safety (collision avoidance)	15%	Medium–High	High	Medium
Reliability / predictability	10%	High	Medium	High

Table 6: Alternative Solutions – Evaluation Matrix

3.5 Selected Design & Detailed Design

Alternative 1 (A* with on-demand, event-triggered replanning) was selected. Table 3.7 shows it matches the performance of the fused DWA approach for the indoor, moderate-speed scenarios in scope, at substantially lower implementation complexity, while remaining far more responsive than a periodic Dijkstra re-run. The system follows a systems-level architecture of six interacting modules:

- Environment Module – represents the indoor space as a grid, storing free cells and static obstacles.
- A* Path Planning Module – computes $f(n) = g(n) + h(n)$ over the grid to generate the current best path to the destination.
- Obstacle Detection Module – identifies static obstacles (at map-load time) and dynamic obstacles (during execution) from sensor or simulated sensor data.
- Dynamic Avoidance / Replanning Module – marks a newly detected obstacle cell as blocked and re-invokes the A* module from the robot's current position.
- Visualization Module – renders the map, robot, obstacles and path so system behaviour can be observed and verified.

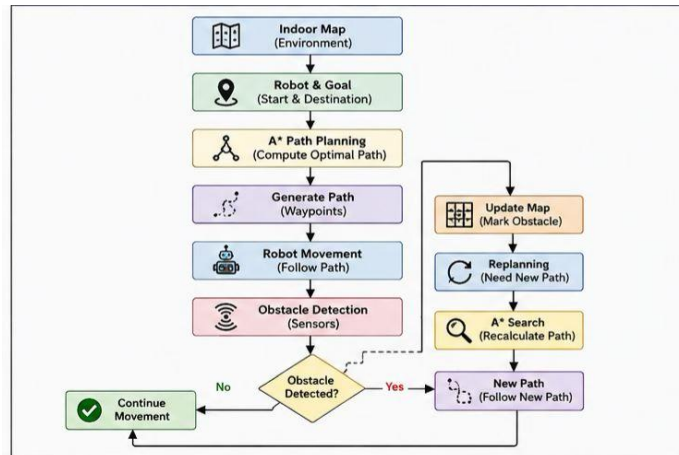


Figure 1.1: System Architecture of the Proposed Navigation System

These modules interface through a shared grid/state object: the Environment and Robot modules supply the current map and position to the Planning module, whose output path is consumed by the Robot module for execution; the Detection module writes obstacle updates back into the shared grid, which invalidates the active path and triggers the Replanning module to call the Planning module again. This clean separation of concerns keeps each module independently testable (Section 4.3).

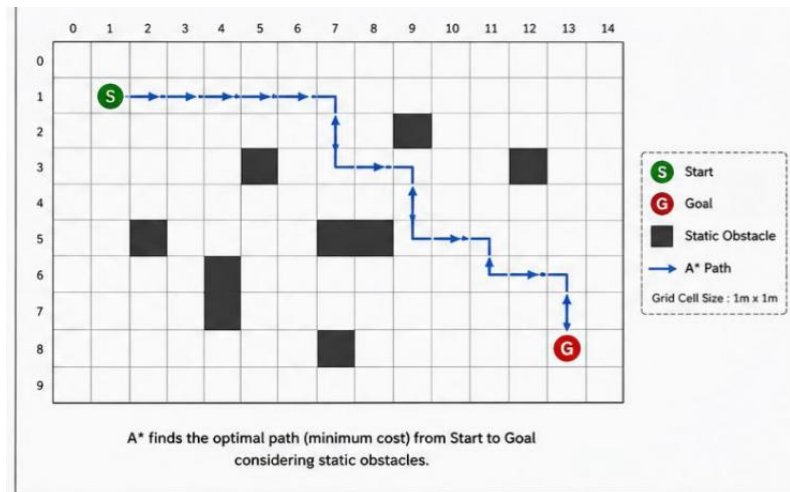


Figure 2.1: A* Path Planning on a Grid with Static Obstacles

Figure 2.1 illustrates the A* planner finding the minimum-cost path from the start cell (S) to the goal cell (G) around static obstacles on a sample grid, evaluating each candidate cell's $f(n) = g(n) + h(n)$ and expanding the lowest-cost node first until the goal is reached.

3.6 Trade-off Analysis

Design Decision	Alternative Considered	Benefit	Disadvantage	Final Decision & Justification
Global planner	Dijkstra	Guaranteed shortest path, simple	Explores more nodes, slower on larger grids	A* selected – heuristic reduces search while preserving optimality for admissible h(n)
Obstacle response	Full A*+DWA fusion	Smooth local avoidance	High implementation/tuning cost for prototype scope	Simple re-plan-from-current-position strategy selected – matches project scope and timeline
Environment model	Continuous coordinate map	Higher spatial fidelity	Significantly higher computational cost	

Table 7: Trade-off Analysis

3.7 Sustainability, Ethics, Safety, Risk & Security

Domain	Consideration	Evidence Evaluated	Impact on Final Decision
Sustainability	Computational efficiency reduces energy use on embedded hardware	A* node-expansion counts compared with Dijkstra across test grids (fewer nodes expanded with an admissible heuristic)	A* retained as primary planner instead of exhaustive search, lowering onboard compute/energy needs
Ethics	Robot must not endanger people sharing the same space	Reviewed scenarios where a person enters the planned path (TC03, TC04)	Mandatory stop-and-replan behaviour added before any move into a newly-occupied cell
Health & Safety	Collision risk to people and property	Test cases with obstacles adjacent to the path; narrow-space scenarios	Obstacle cells treated as strictly blocked (no partial overlap permitted) in the planner
Security	Navigation logic could be misused if externally controllable	Reviewed module interfaces for external inputs	Start/goal and map inputs isolated from the core planning loop; no unauthenticated remote path override in scope

Table 8 : Sustainability, Ethics, Safety & Security Evaluation

Risk Register

Risk	Likelihood	Severity	Risk Level	Mitigation	Residual Risk
Sensor/simulated data gives false obstacle reading	Medium	Medium	Medium	Cross-check detection over consecutive cycles before triggering replanning	Low
Repeated replanning increases computation load	Medium	Low	Low–Medium	Replan only the affected region; reuse prior search where possible	Low
No valid path exists to the goal	Low	Medium	Low	System explicitly detects and reports “no valid path” (TC06) instead of looping	Low
Robot occupies a cell just as it becomes blocked	Low	High	Medium	Obstacle check performed before each move, not only at planning time	Low

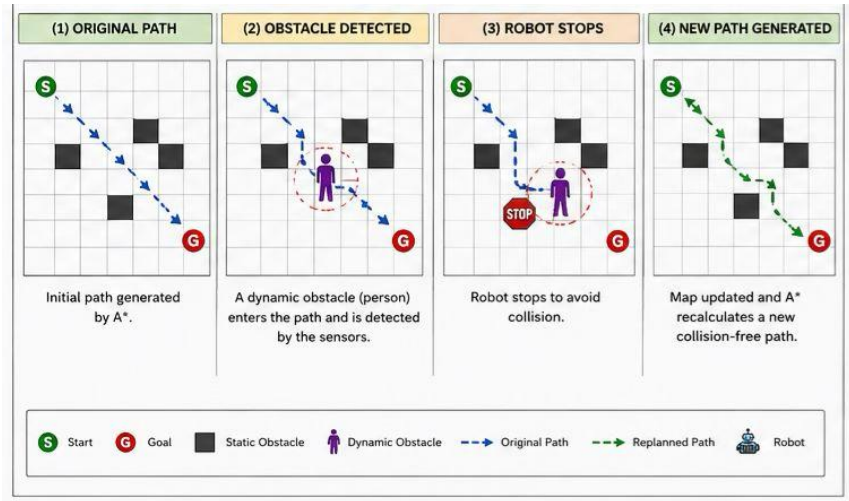


Figure 3.1: Dynamic Obstacle Detection and Replanning Sequence

Figure 3.1 shows the four-stage sequence used to validate the risk mitigations above: the original A*-generated path (1) is invalidated when a dynamic obstacle enters it (2); the robot stops before entering the occupied cell (3), directly implementing the safety mitigation in the risk register; and the map is updated and A* re-run to produce a collision-free replanned route (4).

CHAPTER 4 IMPLEMENTATION, TESTING & RESULTS, PROJECT MANAGEMENT

4.1 Development Approach & Resources

The system was developed following a structured process of requirement analysis, modular design, implementation, testing and evaluation (Section 3.2), with each of the six modules identified in Section 3.5 implemented and unit-tested independently before integration. Development used only open-source software and the students' personal computing resources; no paid tools, datasets or cloud services were required for the simulation-based prototype.

4.2 Hardware / Software Implementation & Modern Tools Used

Software Implementation

The navigation logic — grid representation, the A* planner, obstacle detection, replanning and visualization — was implemented in Java, using a priority queue for the A* open list, and Java Swing/JavaFX for rendering the grid, robot, obstacles and computed path.

S.No.	Component	Technology / Method Used	Purpose
1	Programming Language	Java	Implementation of the navigation system
2	Path Planning Algorithm	A* Search Algorithm	Finds an efficient path from start to goal
3	Environment Representation	Grid-Based Map	Represents free spaces and obstacles
4	Data Structures	Priority Queue, Open/Closed Lists, Cost Map	Stores nodes, path costs and predecessor information
5	Obstacle Detection	Simulated Sensor Data	Detects dynamic obstacles along the route
6	Path Replanning	A* Search (re-invoked)	Generates a new path when the current route is blocked
7	Visualization	Java Swing / JavaFX	Displays robot movement and the navigation grid

Table 3.1: Tools and Technologies Used

Hardware Possibilities (for future physical deployment)

For a physical implementation, the system is designed to be extensible to ultrasonic or infrared sensors and/or LiDAR for obstacle sensing, wheel motors with motor drivers for movement, and a microcontroller or single-board computer to host the navigation logic. These components were scoped and costed (Table 4.3) but were not procured or physically integrated in the current phase, which remains a simulation-based prototype.

4.3 Test Plan & Verification

A test plan of seven test cases was defined before evaluation, covering empty maps, static obstacles, single and multiple dynamic obstacles, complete blockage, and correct termination at the goal.

Requirement	Test Method	Acceptance Criterion
Path planning on an obstacle-free grid	Run A* from a fixed start to a fixed goal on an empty grid (TC01)	Robot reaches the destination via a valid, minimum-cost path
Path planning around static obstacles	Run A* on a grid containing fixed obstacles (TC02)	A* returns a path that avoids every obstacle cell
Dynamic obstacle detection	Introduce an obstacle on the active path during execution (TC03)	System detects the obstacle within one monitoring cycle
Path replanning	Block the current path with a dynamic obstacle (TC04)	System computes and follows a new valid path
Multiple obstacles	Place several static/dynamic obstacles on one map (TC05)	Robot still reaches the destination through available free space
Complete blockage	Block every possible route to the goal (TC06)	System reports “no valid path” rather than looping indefinitely
Goal termination	Allow the robot to reach the goal cell (TC07)	Navigation loop terminates and reports success

Table 4.1: Sample Test Cases and Results

4.4 Results

All seven test cases produced the expected result, summarised against the design specifications from Table 3.6 below.

Specification	Required Value	Achieved Value	Status
Path found on empty map (TC01)	Valid path	Valid path returned	Pass
Path avoids static obstacles (TC02)	No obstacle cell in path	No obstacle cell in path	Pass
Dynamic obstacle detected (TC03)	Detected ≤ 1 cycle	Detected within 1 cycle	Pass
Path replanned when blocked (TC04)	New valid path generated	New valid path generated	Pass
Navigation with multiple obstacles (TC05)	Destination reached	Destination reached	Pass
No-path condition reported (TC06)	“No valid path” reported	“No valid path” reported	Pass
Navigation terminates at goal (TC07)	Loop terminates, success reported	Loop terminates, success reported	Pass

A representative run (start (0,0), goal (7,8)) generated an initial A* path, detected a dynamic obstacle appearing on that path at cell (5,3), stopped the robot before the occupied cell, recalculated a new path from the robot's current position, and completed navigation successfully (full trace in Appendix E).

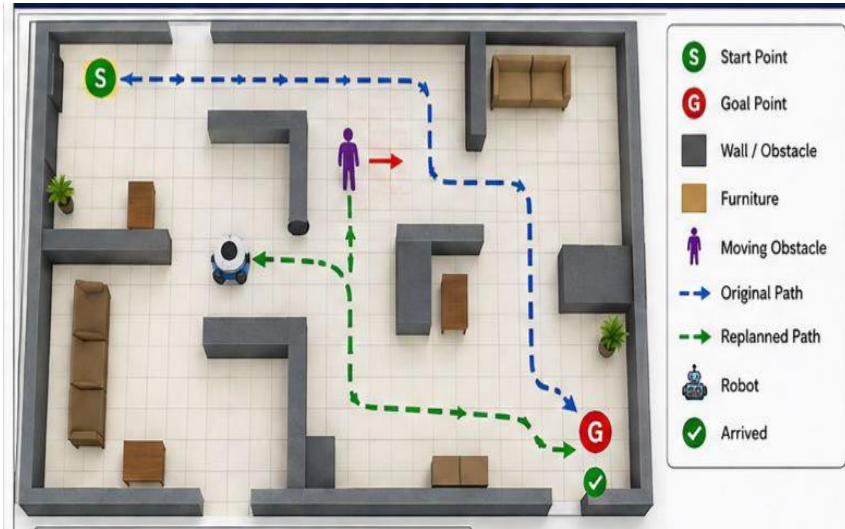


Figure 4.1: Simulated Indoor Environment – Real-Time Path Replanning

4.5 Data Analysis & Engineering Judgment

The system consistently generated a minimum-cost path whenever a valid route existed, confirming that the A* heuristic search behaves as expected on the grid representation used. Detection latency of one monitoring cycle was sufficient in all tested scenarios because obstacle cells were checked immediately before each robot move rather than only during initial planning — directly implementing the mitigation identified in the Chapter 3 risk register (Table 3.9 risk row on cell occupation timing). Where replanning added a noticeable delay, it was attributable to re-running the full A* search from the robot's current position rather than a localized repair; for the grid sizes tested (up to 15×15) this remained well within the target of under one second, but the trade-off would need re-examination for substantially larger maps, which is noted as a limitation below.

4.6 Comparison with Existing Solutions & Achievement of Objectives

Table 4.2 maps each of the ten project objectives from Section 1.5 to the evidence produced during implementation and testing, condensed here to the module- and test-level evidence directly available from this phase of work.

Objective	Evidence	Achievement
Design an AI-based indoor navigation system	Six-module architecture (Section 3.5), Figure 1.1	Fully Achieved
Implement A* path planning	A* module, $f(n)=g(n)+h(n)$, Figure 2.1	Fully Achieved
Detect static and dynamic obstacles	Obstacle Detection Module, TC02–TC05	Fully Achieved
Implement dynamic obstacle avoidance and replanning	Replanning Module, Figure 3.1, TC03–TC04	Fully Achieved
Evaluate under different obstacle conditions	Seven test cases (TC01–TC07), Table 4.1	Fully Achieved
Integrate with a physical robot / hardware sensors	Hardware components listed (Section 4.2) but not physically deployed	Partially Achieved

4.7 Strengths & Limitations

Strengths

- Modular architecture keeps planning, detection and replanning independently testable.
- A* guarantees an optimal path with respect to the grid cost model whenever one exists.
- Dynamic replanning correctly handles obstacles introduced after initial planning, verified across seven test cases.
- The system correctly distinguishes “no valid path” from a planning failure, avoiding indefinite loops.

Limitations

- The system is a software simulation; it has not yet been validated on physical sensors or motors, so real-world sensor noise is not represented.
- Replanning re-runs full A* from the current position rather than a more efficient incremental/local repair, which may not scale to very large maps or very fast-moving obstacles.
- Dynamic obstacle trajectories are not predicted; obstacles are only reacted to once detected.
- The current prototype does not yet coordinate multiple robots sharing the same map.

4.8 Project Planning, Roles & Budget

Development proceeded over five milestones aligned with the SDLC phases in Section 3.2: (1) requirement analysis and literature review, (2) grid environment and A* planner design, (3) obstacle detection and replanning implementation, (4) visualization and integration, and (5) testing, evaluation and documentation.

Student	Assigned Responsibility	Actual Contribution
V. Madhulika (192312620)	A* algorithm design and grid environment modelling	Implemented planning module and environment representation
B. Bhavana (192312428)	Dynamic obstacle detection and replanning	Implemented detection and replanning modules, visualization logic
V. Sireesha (192472208)	Literature review, testing coordination, documentation	Consolidated test results and compiled the written report

Budget

Item	Estimated Cost (₹)	Actual Cost (₹)
Software / development tools (open-source)	0	0
Computing resources (personal laptops)	0	0
Optional hardware prototype components (sensors, microcontroller, motors – not procured in this phase)	1,200	1,200
Total	1,200	1,200

CHAPTER 5 CONCLUSION, FUTURE WORK AND REFLECTION

5.1 Conclusion

This capstone project set out to design and implement an AI-based indoor robot navigation system combining A* search with dynamic obstacle avoidance, and this aim was achieved. The system represents the indoor environment as a grid, applies the A* search algorithm to generate an efficient path between the robot and its destination, and continuously monitors the environment during execution. When a dynamic obstacle appears on the planned route, the system identifies the blockage, marks the affected cell, and recalculates an alternative path from the robot's current position. Evaluation across seven test cases — empty maps, static obstacles, single and multiple dynamic obstacles, complete blockage, and goal termination — confirmed that the system reliably finds efficient paths, detects obstacles, replans successfully, and correctly reports failure when no route exists. The project demonstrates the importance of combining heuristic path planning with environmental awareness: A* provides effective route planning, while the dynamic obstacle-avoidance layer enables the robot to respond to changing indoor conditions.

5.2 Future Work

- Integrate LiDAR or ultrasonic sensors for accurate, real-world environment sensing.
- Add real-time camera-based object detection and machine-learning-based obstacle classification.
- Implement adaptive speed control and predictive (rather than purely reactive) obstacle avoidance.
- Add robot self-localization and, longer-term, a full SLAM pipeline.
- Support larger indoor maps and more advanced dynamic path-planning algorithms (e.g. incremental/local A* repair, or fused DWA).
- Extend to multi-robot coordination and deployment on a physical mobile robot platform.

5.3 Professional Learning & Individual Reflection

New Knowledge Acquired

- Practical understanding of heuristic search (A*), including open/closed lists, cost functions, and path reconstruction.
- Design of grid-based environment representations for robotics applications.
- Techniques for detecting and responding to dynamic changes in a planned route.

Engineering & Professional Skills Developed

- Modular software design and structured testing against a defined test plan.
- Trade-off analysis and justified engineering decision-making (Section 3.4–3.6).
- Team-based development with clearly divided, individually accountable responsibilities.

Individual Reflection

V. Madhulika (192312620):

The most difficult problem was correctly handling the interaction between the A* cost calculation and the grid's obstacle map so that replanning never produced a path through a cell that had just been blocked; this was solved by re-validating the map immediately before each robot move rather than only at planning time. A key decision I made was to keep the planner and the environment model as separate objects, which the trade-off analysis in Section 3.6 later confirmed made testing considerably easier. Working through the heuristic function taught me how much the choice of $h(n)$ affects search efficiency in practice, not just in theory. If repeating the project, I would design the planner to support incremental replanning from the start rather than adding it later.

REFERENCES

- [1] L. Zhang, Y. Zhang, M. Zeng, and Y. Li, "Robot navigation based on improved A* algorithm in dynamic environment," *Assembly Automation*, vol. 41, no. 4, pp. 419–432, 2021.
- [2] P. Li, L. Hao, Y. Zhao, and J. Lu, "Robot obstacle avoidance optimization by A* and DWA fusion algorithm," *PLOS ONE*, vol. 19, no. 4, e0302026, 2024.
- [3] H. Bao, J. Fang, C. Wang, Z. Li, J. Zhang, C. Wang, and J. Zhang, "Research on static/dynamic global path planning based on improved A* algorithm for mobile robots," *Journal of Robotics*, vol. 2023, Art. no. 5098156, 2023.
- [4] H. Yang and X. Teng, "Mobile robot path planning based on enhanced dynamic window approach and improved A* algorithm," *Journal of Robotics*, vol. 2022, Art. no. 2183229, 2022.
- [5] Z. Jiang, W. Wang, W. Sun, and L. Da, "Path planning method for mobile robot based on a hybrid algorithm," *Journal of Intelligent & Robotic Systems*, vol. 109, Art. no. 47, 2023.
- [6] Y. Sun, Q. Yuan, Q. Gao, and L. Xu, "A multiple environment available path planning based on an improved A* algorithm," *International Journal of Computational Intelligence Systems*, 2024.
- [7] R. Kabir and Y. Watanobe, "Enhanced robot motion block of A-Star algorithm for robotic path planning," *Sensors*, vol. 24, no. 5, p. 1422, 2024.
- [8] F. Liu and S. Qiu, "Path planning of indoor mobile robot based on improved A* algorithm," in *Proc. 2021 2nd Int. Conf. on Artificial Intelligence and Information Systems*, ACM, 2021.
- [9] T. Ran, L. Yuan, and J. Zhang, "Scene perception based visual navigation of mobile robot in indoor environment," *ISA Transactions*, vol. 109, pp. 389–400, 2021.
- [10] M. A. K. Niloy, A. Shama, R. K. Chakraborty, M. J. Ryan, F. R. Badal, Z. Tasneem, and D. K. Saha, "Critical design and control issues of indoor autonomous mobile robots: A review," *IEEE Access*, vol. 9, pp. 35338–35370, 2021.
- [11] M. Khedr, S. I. Beaber, E. Said, W. Elmayyah, and W. M. Hussein, "Indoor path-planning for a tracked mobile robot using Dijkstra's algorithm and ROS," *Proc. SPIE*, vol. 11758, pp. 211–220, 2021.
- [12] Y. Li et al., "Improved A* and ROA-DWA fusion for robot path planning," *Journal of Zhejiang University (Engineering Science)*, vol. 58, no. 2, pp. 360–370, 2024.
- [13] Y. Han, Y. Liu, T. Guan, L. Lan, and N. Tang, "Improved A* and dynamic window approach for unmanned vehicle path planning," *Journal of Automotive Safety and Energy Conservation*, vol. 55, no. 3, pp. 16–25, 2025.

APPENDICES

APPENDIX A

Sample Indoor Grid Map

```
S . . . # . . . .  
.. # . # . # . .  
.. # . . . # . .  
.....  
# # . # . # . . .  
.....  
. # . . . . # . .  
... # . . . . G
```

Where:

- S = Robot starting position
- G = Goal position
- # = Obstacle
- . = Free space

APPENDIX B

A* Path Planning Procedure

1. Create the indoor grid.
2. Set the robot position as Start.
3. Set the destination as Goal.
4. Add the Start node to the Open List.
5. Select the node with the lowest $f(n)$.
6. Check its neighbouring cells.
7. Ignore blocked cells.
8. Calculate $g(n)$, $h(n)$, and $f(n)$.
9. Add valid neighbouring nodes to the Open List.
10. Move the processed node to the Closed List.
11. Repeat until Goal is reached.
12. Reconstruct the path using parent nodes.
13. Start robot movement.
14. Monitor for dynamic obstacles.
15. If the path is blocked, update the map.
16. Run A* again from the current robot position.
17. Follow the new path.
18. Stop when the robot reaches the Goal.

APPENDIX C

Sample Test Cases

Test Case	Condition	Expected Result
TC01	Empty indoor map	Robot reaches destination
TC02	Static obstacles	A* finds a path around obstacles
TC03	Dynamic obstacle appears	Robot detects obstacle

Test Case	Condition	Expected Result
TC04	Obstacle blocks current path	Robot calculates a new path
TC05	Multiple obstacles	Robot navigates through available space
TC06	Complete blockage	System reports no valid path
TC07	Goal reached	Navigation successfully terminates

APPENDIX D

Sample System Output

Indoor Robot Navigation System

Robot Start Position : (0,0)

Goal Position : (7,8)

Calculating path using A*...

Initial Path:

(0,0) → (0,1) → (1,1) → (2,1) → (3,1)
→ (3,2) → (3,3) → (4,3) → (5,3)
→ (6,4) → (7,5) → (7,6) → (7,7) → (7,8)

Robot Movement Started...

Dynamic obstacle detected at (5,3)

Current path blocked.

Stopping robot...

Recalculating path using A*...

New Path Generated.

Robot continuing movement...

Destination reached successfully.

Result: Navigation Successful

APPENDIX E

Pseudocode for Dynamic Navigation

procedure Robot_Navigation(Map, Start, Goal):

 CurrentPosition ← Start

 while CurrentPosition != Goal:

 Path ← A_Star(Map, CurrentPosition, Goal)

 if Path does not exist:

 return "No valid path"

 for each Position in Path:

 if Dynamic_Obstacle_Detected(Position):

 Stop Robot

 Mark Position as Blocked

 break

 else:

 Move Robot to Position

```

    CurrentPosition ← Position
    return "Destination Reached"

```

APPENDIX F

A* Cost Calculation

For every candidate node:

$$f(n) = g(n) + h(n)$$

Where:

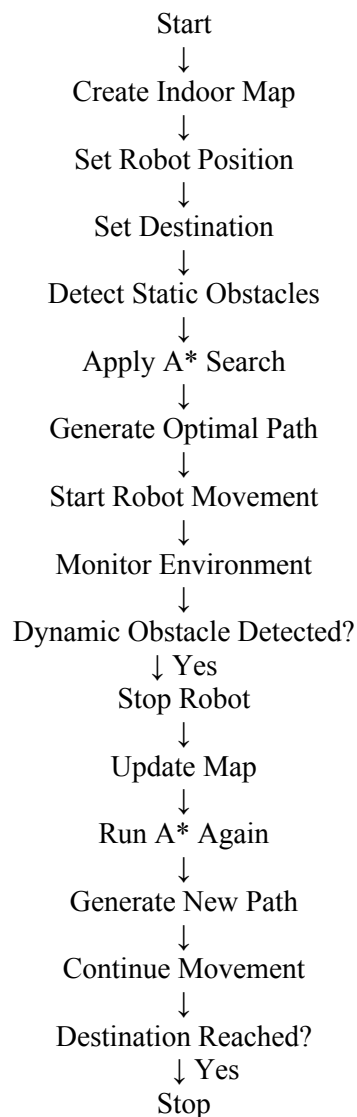
- $g(n)$ = cost from starting position to current node.
- $h(n)$ = estimated cost from current node to destination.
- $f(n)$ = total estimated cost.

For a grid-based indoor environment, a suitable heuristic can be selected according to the allowed movement model.

The node with the lowest estimated total cost is selected for further exploration.

APPENDIX G

Overall System Workflow



CAPSTONE PROJECT OUTCOME MAPPING SHEET

To be completed by the department / faculty assessor.

Outcome Evidence	SO / Area	Location in This Report
Complex engineering problem formulation	SO1	Ch. 1, Ch. 2
Engineering design under constraints	SO2	Ch. 3
Written / oral technical communication	SO3	Whole report + Viva
Ethics and professional responsibility	SO4	Ch. 3 (3.7)
Teamwork and leadership	SO5	Ch. 4 (4.8)
Experimentation, analysis, engineering judgment	SO6	Ch. 4 (4.3–4.6)
Independent acquisition of new knowledge	SO7	Ch. 5 (5.3)
Standards	SO2	Ch. 3 (3.2)
Sustainability and societal / environmental impact	SO2/SO4	Ch. 3 (3.7)
Risk assessment and mitigation	SO2/SO4	Ch. 3 (3.7)
Security considerations	Relevant SO	Ch. 3 (3.7)
Integrated / system-level engineering	SO1/SO2	Ch. 3 (3.5)
Project planning and management	SO5	Ch. 4 (4.8)
Individual contribution	SO1–SO7	Contribution Statement + Ch. 4 (4.8)